Universidade Estadual de Campinas Instituto de Matemática, Estatística e Computação Científica

ME607 – Séries Temporais

Semestre 01/2026 – Trabalho 1

Redes de Memória de Longo Prazo (LSTM) Aplicadas a Séries Temporais

Uma Discussão Detalhada e Crítica

Ilustrada com o Conjunto International Airline Passengers (1949–1960)

Natalie Janine Merk - RA: 300332

April 17, 2026

Abstract

Este trabalho apresenta uma discussão detalhada e crítica das Redes de Memória de Longo Prazo (*Long Short-Term Memory*, LSTM), introduzidas por Hochreiter and Schmidhuber [7] e posteriormente consolidadas como uma das arquiteturas de aprendizado profundo mais influentes para dados sequenciais. As redes LSTM pertencem à família das redes neurais recorrentes (RNN) e foram concebidas especificamente para superar a limitação das RNNs clássicas no aprendizado de dependências temporais de longo alcance. O modelo é aplicado ao conjunto de dados *International Airline Passengers* [2, Series G], um benchmark clássico de séries temporais com 144 observações mensais, tendência crescente e sazonalidade anual marcada. São discutidos a arquitetura do modelo, o processo de treinamento, os resultados obtidos, as vantagens, as limitações e as principais extensões e melhorias propostas na literatura recente.

Contents

1 Introdução **3**

2 Redes Neurais Recorrentes e o Problema do Gradiente Evanescente **3**
2.1 Redes Neurais Recorrentes (RNN) 3
2.2 O Problema do Gradiente Evanescente 4

3 Arquitetura da Rede LSTM **4**
3.1 A Solução: Estado da Célula e Portas de Controle 4
3.2 Por que o Estado da Célula Resolve o Gradiente Evanescente? 5
3.3 Configuração de Arquiteturas LSTM 6

4 Conjunto de Dados **6**
4.1 International Airline Passengers 6
4.2 Preparação dos Dados 9

5 Treinamento do Modelo **10**
5.1 Configuração da Arquitetura 10
5.2 Evolução do Treinamento 11

6 Resultados **12**
6.1 Previsão na Série Completa 12
6.2 Previsão no Conjunto de Teste 12

7 Vantagens do Modelo LSTM **13**

8 Desvantagens e Limitações **14**

9 Como Melhorar o Modelo **15**

9.1 Janela Deslizante Maior 15

9.2 Arquiteturas Mais Profundas e Regularização 15

9.3 LSTM Bidirecional 16

9.4 Mecanismos de Atenção e Transformers 16

10 Adequação de Modelos: LSTM vs. Métodos Clássicos **17**

11 Conclusão **18**

Referências **19**

Arquivos Supplementares **21**

1. Introdução

A previsão de séries temporais é uma das tarefas de maior importância em estatística aplicada, com aplicações em economia, finanças, meteorologia, engenharia e ciências da saúde, entre muitas outras áreas. Os métodos clássicos abordados na disciplina — modelos ARIMA, modelos sazonais, suavização exponencial, funções de transferência e modelos ARMAX, entre outros — fornecem um arcabouço teórico rigoroso, fundamentado na teoria de processos estocásticos e continuam a ser amplamente utilizados na prática [9, 10].

Nas últimas décadas, contudo, o avanço do aprendizado profundo (*deep learning*) introduziu uma abordagem distinta: em vez de especificar a estrutura estocástica do processo gerador de dados, treina-se uma função altamente flexível para mapear sequências passadas a valores futuros. No contexto de dados sequenciais e temporais, as Redes Neurais Recorrentes (RNN) constituem a arquitetura natural, e as LSTMs são uma variante muito estudada e bem-sucedida desse paradigma [2, 3].

Este relatório tem como objetivo descrever e discutir criticamente o modelo LSTM, situando-o no contexto da análise de séries temporais, com ênfase em seus fundamentos, mecanismos internos, comportamento prático e limitações. A ilustração empírica é feita com o conjunto *International Airline Passengers*, utilizando experimentos próprios cujos resultados são apresentados e interpretados nas seções seguintes.

2. Redes Neurais Recorrentes e o Problema do Gradiente Evanescente

2.1. Redes Neurais Recorrentes (RNN)

Uma RNN processa uma sequência $x_1, x_2, \ldots, x_T$ mantendo um estado oculto $h_t \in \mathbb{R}^n$ que é atualizado a cada passo de tempo:

$$h_t = \tanh\big(W_h\, h_{t-1} + W_x\, x_t + b_h\big), \tag{1}$$

$$\hat{y}_t = W_y\, h_t + b_y. \tag{2}$$

A ideia é que h_t acumule informação sobre todo o histórico da sequência até o instante t. Na prática, porém, esse mecanismo simples enfrenta uma dificuldade fundamental: o **problema do gradiente evanescente** [3, Capítulo 19].

2.2. O Problema do Gradiente Evanescente

Durante o treinamento via retropropagação no tempo (*Backpropagation Through Time*, BPTT), o gradiente da função de perda em relação aos parâmetros da rede (W_x, W_h, b_h, entre outros) é calculado recursivamente ao longo dos passos de tempo. A cada passo, o gradiente é multiplicado pela matriz de pesos W_h e pelo termo derivado da função de ativação, $(1 - h_t^2)$.

Como $h_t = \tanh(a_t) \in (-1, 1)$ para todo $a_t \in \mathbb{R}$, segue que $(1 - h_t^2) \in (0, 1)$ para todo $t \in \mathbb{N}$. Assim, o produto sucessivo desses fatores implica uma diminuição progressiva da magnitude do gradiente. Quanto maior a profundidade temporal da RNN, maior o número de termos $(1 - h_t^2)$ envolvidos, e menor tende a ser o gradiente.

Ao mesmo tempo, se o maior valor singular de W_h for menor que 1, os gradientes decaem exponencialmente com a distância temporal; se for maior que 1, eles explodem. Em ambos os casos, o treinamento torna-se instável ou ineficaz para dependências de longo alcance.

Na prática, isso implica que uma RNN simples não consegue aprender dependências temporais de longo prazo. Para ciclos de maior período, isso é insuficiente. Essa limitação motivou o desenvolvimento da arquitetura LSTM [3, Capítulo 17].

3. Arquitetura da Rede LSTM

3.1. A Solução: Estado da Célula e Portas de Controle

A LSTM introduz dois vetores de estado:

- O **estado da célula** $c_t \in \mathbb{R}^n$, que funciona como uma memória de longo prazo transportada ao longo da sequência com interações predominantemente lineares;
- O **estado oculto** $h_t \in \mathbb{R}^n$, que é utilizado para a remoção e a adição de informação ao estado da célula.

O acesso e a atualização dessas memórias são controlados por três **portas** (*gates*) – Porta de Esquecimento, Porta de Entrada, Porta de Saída – cada uma produzindo um vetor de valores em $(0, 1)$ através de funções sigmoide:

(1) Porta de Esquecimento (*Forget Gate*) — decide que fração da memória anterior deve ser descartada:

$$f_t = \sigma\big(W_f\,[h_{t-1},\, x_t] + b_f\big). \tag{3}$$

(2) Porta de Entrada (*Input Gate*) — decide que novas informações serão escritas na

célula:

$$i_t = \sigma\big(W_i\,[h_{t-1},\,x_t] + b_i\big), \tag{4}$$

$$\tilde{c}_t = \tanh\big(W_c\,[h_{t-1},\,x_t] + b_c\big). \tag{5}$$

(3) Atualização do Estado da Célula — combina o esquecimento e a nova informação:

$$c_t = f_t \odot c_{t-1} \;+\; i_t \odot \tilde{c}_t. \tag{6}$$

(4) Porta de Saída (*Output Gate*) — decide que parte da célula será exposta como estado oculto:

$$o_t = \sigma\big(W_o\,[h_{t-1},\,x_t] + b_o\big), \tag{7}$$

$$h_t = o_t \odot \tanh(c_t). \tag{8}$$

Nestas equações, $\sigma(\cdot)$ é a função sigmoide, $\tanh(\cdot)$ é a tangente hiperbólica, $\odot$ denota o produto elemento a elemento (Hadamard) e $[h_{t-1},\,x_t]$ é a concatenação dos dois vetores. As matrizes de pesos W_f, W_i, W_c, W_o e os vieses b_f, b_i, b_c, b_o são os parâmetros aprendidos durante o treinamento [3, Capítulo 17, 6, Capítulo 11.6].

3.2. Por que o Estado da Célula Resolve o Gradiente Evanescente?

A equação (6) é a chave. Para entender por que o gradiente não se evanece, aplica-se a regra da cadeia à atualização do estado da célula. Definindo a função de erro total como $\mathcal{L}$, o gradiente em relação a c_{t-1} é:

$$\frac{\partial \mathcal{L}}{\partial c_{t-1}} = \frac{\partial \mathcal{L}}{\partial c_t} \cdot \frac{\partial c_t}{\partial c_{t-1}} = \frac{\partial \mathcal{L}}{\partial c_t} \cdot f_t, \tag{9}$$

pois $\partial c_t / \partial c_{t-1} = f_t$ diretamente de (6), dado que o termo $i_t \odot \tilde{c}_t$ não depende de c_{t-1}. Generalizando por retropropagação no tempo até k passos:

$$\frac{\partial \mathcal{L}}{\partial c_{t-k}} = \frac{\partial \mathcal{L}}{\partial c_t} \cdot \prod_{j=1}^{k} f_{t-j+1}. \tag{10}$$

Este produto envolve apenas os *valores escalares* das portas de esquecimento, e não produtos de matrizes de pesos como na RNN simples. Quando $f_t \approx 1$, o gradiente é propagado sem atenuação; quando $f_t \approx 0$, é bloqueado intencionalmente – o modelo aprendeu que esse contexto passado não é relevante. Esta assimetria é fundamental: o evanescimento deixa de ser uma patologia e passa a ser um comportamento *aprendido e controlado*. A rede aprende *quando* lembrar, *quando* esquecer e *quando* atualizar a sua memória diretamente a partir dos dados [7].

3.3. Configuração de Arquiteturas LSTM

Um dos aspectos fundamentais a definir é o formato de entrada do modelo, normalmente especificado como uma sequência de comprimento fixo, multiplicada pelo número de variáveis explicativas (covariáveis) presentes nos dados por observação.

Outro parâmetro essencial é o número de unidades na camada LSTM, anteriormente denotado como n, o qual define a dimensionalidade do estado oculto da rede. Valores mais elevados aumentam a capacidade do modelo de representar dinâmicas temporais complexas, porém também implicam maior custo computacional e maior risco de sobreajuste, uma vez que o número de parâmetros treináveis cresce proporcionalmente.

Além disso, é necessário definir se a camada LSTM deve retornar a sequência completa de estados ocultos ou apenas o último estado. Essa escolha no paquete Keras do Python é controlada pelo parâmetro *return_sequences*.

Quando *return_sequences = True*, a LSTM produz uma saída em cada passo temporal, gerando uma sequência de estados ocultos. Nesse caso, a saída mantém a mesma dimensão temporal da entrada, podendo ser interpretada como uma representação aprendida da sequência ao longo do tempo. Essa configuração é necessária quando camadas subsequentes — por exemplo, outra camada LSTM — exigem uma entrada sequencial, ou quando o objetivo do modelo é realizar previsões em cada instante temporal.

Por outro lado, quando *return_sequences = False*, apenas o último estado oculto é retornado. Esse estado corresponde a um vetor de dimensão fixa que resume toda a sequência de entrada, sendo então utilizado em tarefas que exigem uma única predição por sequência, como a previsão ponto a ponto de séries temporais.[4].

4. Conjunto de Dados

4.1. International Airline Passengers

O conjunto de dados *International Airline Passengers* contém o total mensal de passageiros internacionais de aviação, em milhares, de janeiro de 1949 a dezembro de 1960, totalizando 144 observações. A Figura [2, Series G] apresenta a série completa.

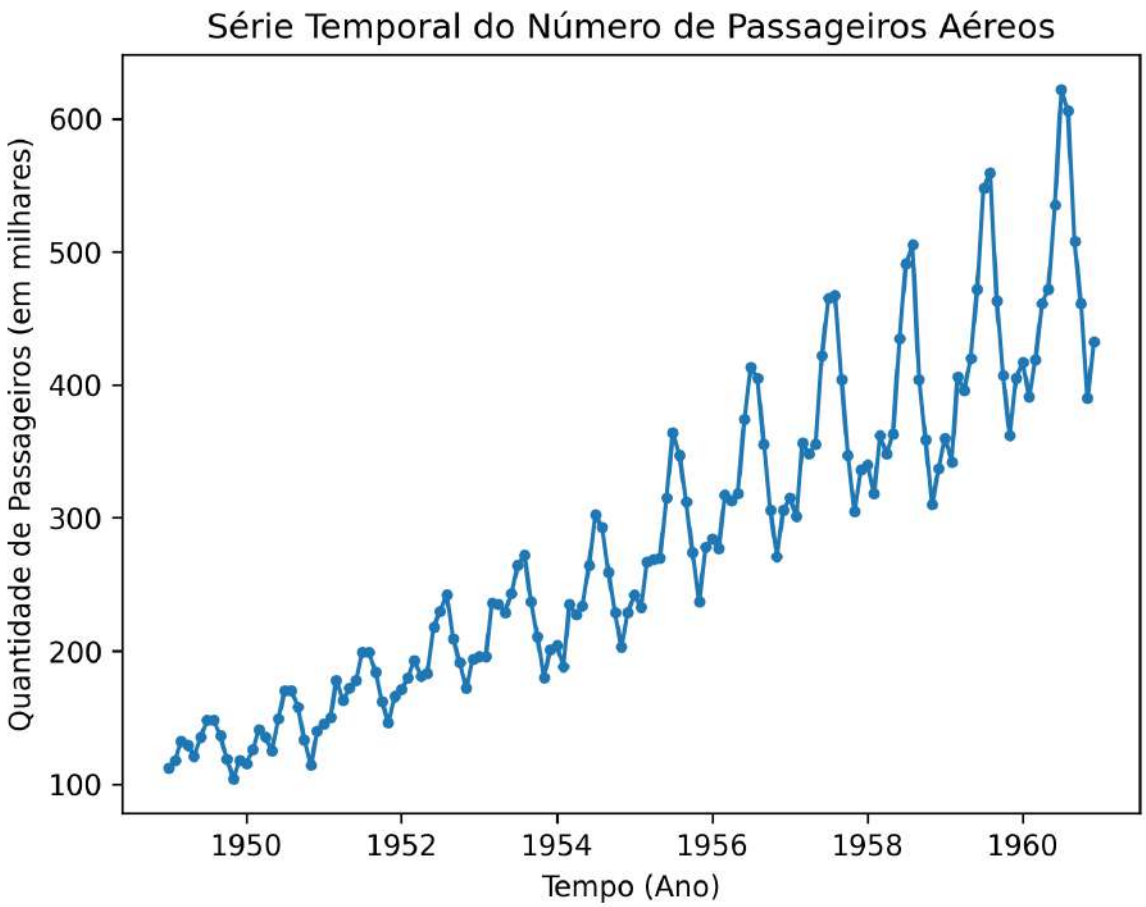


Figure 1. Série temporal do número de passageiros aéreos internacionais (1949–1960). Observa-se uma tendência crescente clara e sazonalidade anual com amplitude que cresce com o nível da série.

As características principais da série são:

- **Tendência crescente** ao longo de todo o período, com o número de passageiros mais que triplicando entre 1949 e 1960;
- **Sazonalidade anual marcada** de estrutura multiplicativa: os picos ocorrem nos meses de verão do hemisfério norte (julho–agosto) e os vales no inverno (novembro–fevereiro), com amplitude proporcional ao nível da série;
- **Ausência de valores faltantes** e de intervenções externas identificáveis, o que torna a série um benchmark limpo e bem controlado.

Para aprofundar a análise da estrutura da série, a Figura 2 apresenta sua decomposição clássica em componentes de tendência, sazonalidade e resíduo, assumindo um modelo multiplicativo.

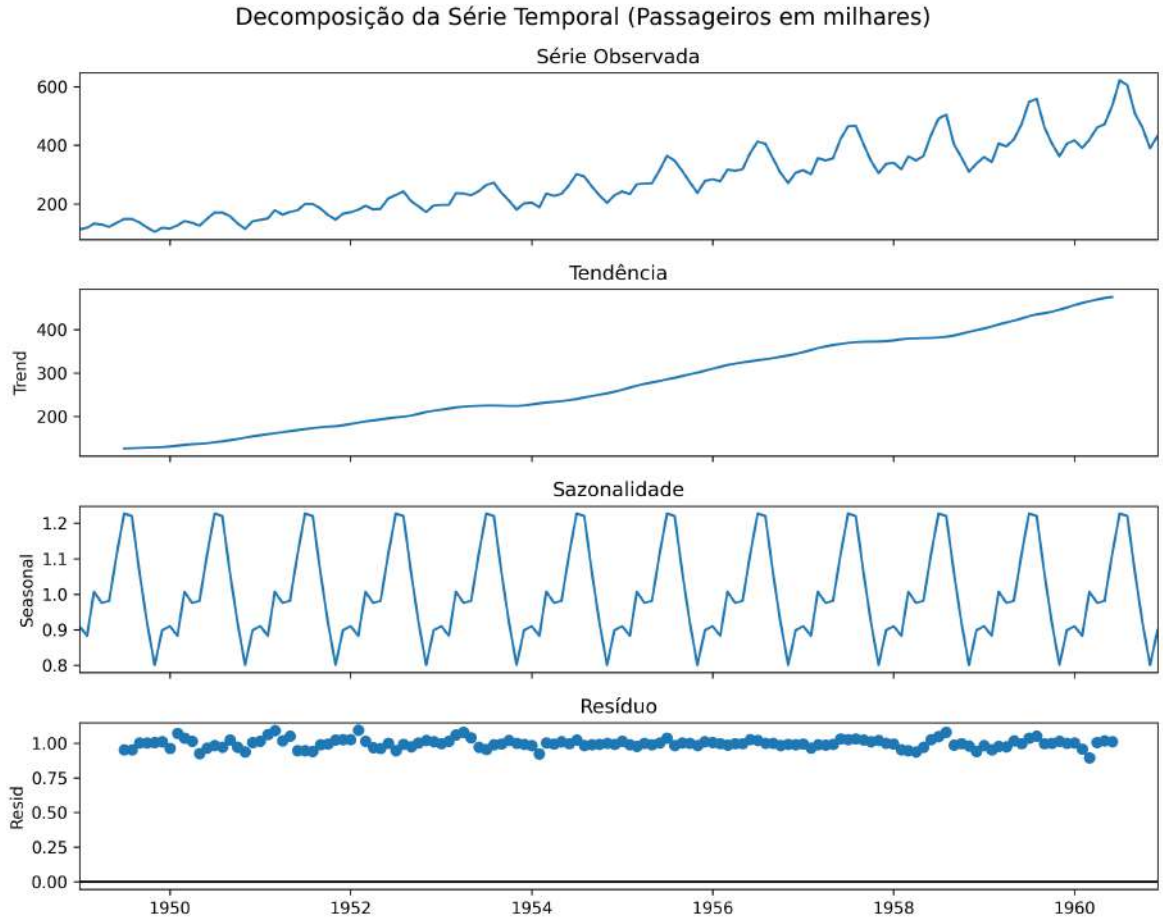
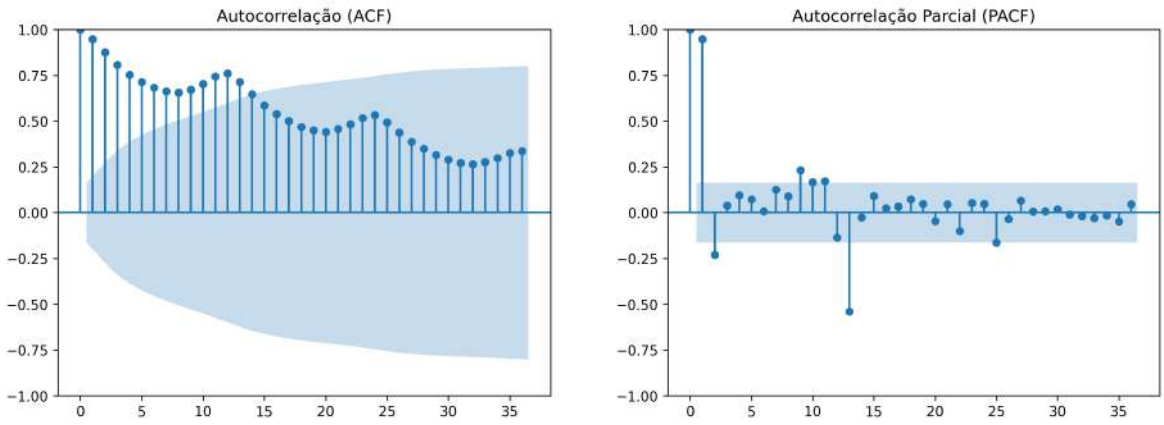


Figure 2. Decomposição multiplicativa da série temporal. Observa-se uma tendência crescente suave, uma componente sazonal estável ao longo do tempo e resíduos relativamente pequenos em comparação com o nível da série.

A decomposição confirma a presença de uma tendência de longo prazo claramente crescente, bem como uma sazonalidade anual regular. A estabilidade do padrão sazonal ao longo do tempo reforça a hipótese de uma estrutura multiplicativa, na qual a amplitude das flutuações sazonais aumenta proporcionalmente ao nível da série.

Além disso, a dependência temporal da série pode ser analisada por meio das funções de autocorrelação (ACF) e autocorrelação parcial (PACF), apresentadas nas Figuras 3a e 3b. As funções foram calculadas considerando até 36 defasagens, de modo a capturar múltiplos ciclos sazonais anuais.



(a) Função de autocorrelação (ACF) **(b)** Função de autocorrelação parcial (PACF)

Figure 3. Funções de autocorrelação (ACF) e autocorrelação parcial (PACF) da série de passageiros.

4.2. Preparação dos Dados

Antes de alimentar a rede LSTM, os dados foram submetidos às seguintes etapas de pré-processamento:

- **Normalização min-max:** a série foi escalada para o intervalo $[0, 1]$ com a transformação $\tilde{y}_t = (y_t - y_{\min})/(y_{\max} - y_{\min})$. Essa etapa é essencial para estabilizar o treinamento, pois as ativações sigmoide e tanh das portas operam eficientemente nesse intervalo;
- **Construção de janela deslizante:** a série foi reorganizada como um problema de aprendizado supervisionado, com janela de comprimento $L = 12$ passos (equivalente a um ano de dados) como entrada e o valor seguinte como alvo;
- **Divisão temporal:** os dados normalizados foram divididos em três conjuntos disjuntos e temporalmente ordenados, conforme ilustrado na Figura 4. Essa divisão foi realizada previamente à aplicação da técnica de *sliding window*, de modo a preservar a estrutura temporal das sequências e evitar vazamento de informação entre os conjuntos.

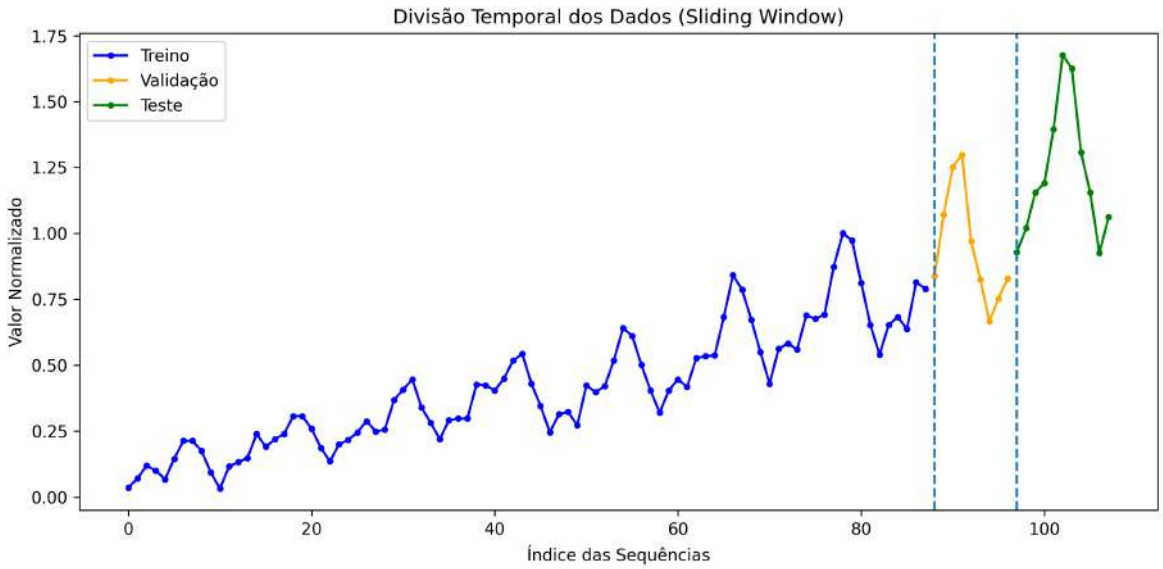


Figure 4. Divisão temporal dos dados normalizados em conjuntos de treino (azul), validação (laranja) e teste (verde). A divisão respeita a ordem cronológica da série para evitar o vazamento de informação futura (*data leakage*).

A Figura 4 evidencia a estrutura de divisão adotada, com os três segmentos claramente separados no tempo. Note-se que os valores normalizados do conjunto de teste atingem os máximos da série (próximo de 1,0), correspondendo ao pico de passageiros observado em 1960–1961.

5. Treinamento do Modelo

5.1. Configuração da Arquitetura

O modelo LSTM utilizado neste experimento foi configurado com:

- Uma camada LSTM com 50 unidades ocultas;
- Uma camada densa de saída com uma única unidade e ativação linear;
- Otimizador: Adam;
- Função de perda: erro quadrático médio (EQM/MSE);
- Tamanho do lote (*batch size*): 8;
- Número máximo de épocas: 50;
- Tamanho da janela temporal (*time step*): 12.

5.2. Evolução do Treinamento

A Figura 5 apresenta a evolução das perdas de treino e validação ao longo das 50 épocas de treinamento.

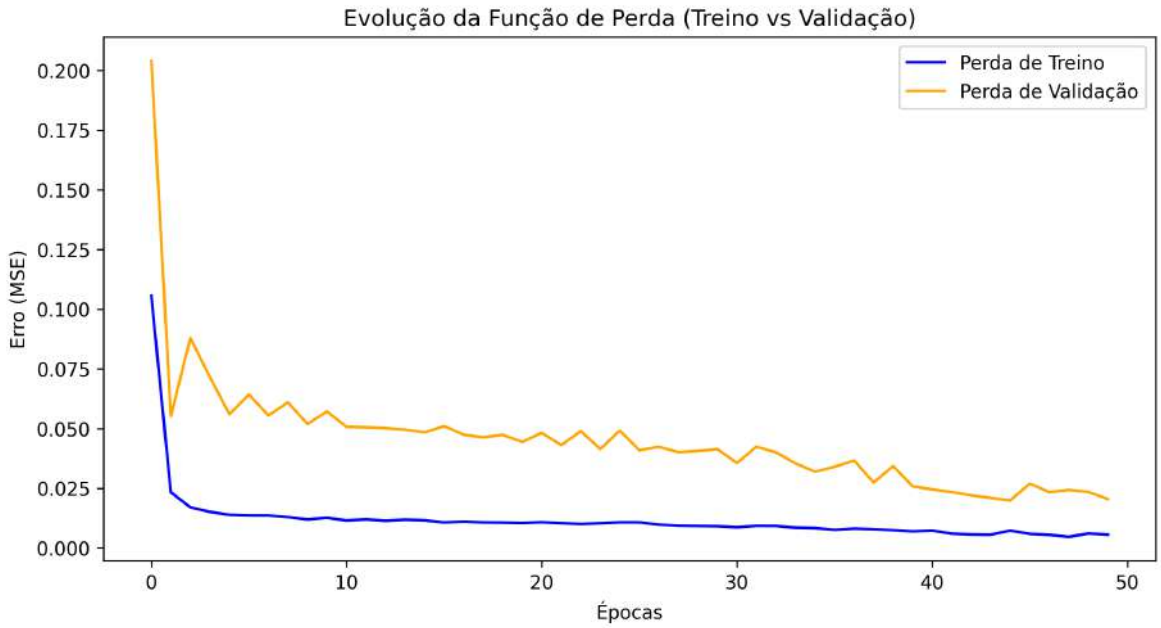


Figure 5. Evolução da função de perda (MSE) durante o treinamento. A perda de treino (azul) decresce monotonicamente, atingindo valores próximos de zero. A perda de validação (laranja) estabiliza-se em torno de 0,013–0,020 após a época 5, com oscilações residuais.

Podem-se identificar três fases distintas no gráfico de treinamento:

1. **Convergência rápida inicial (épocas 0–5):** ambas as perdas diminuem acentuadamente. O modelo aprende rapidamente a tendência geral e o nível médio da série;

2. **Refinamento gradual (épocas 5–20):** a perda de treino continua a cair lentamente; a perda de validação estabiliza-se com pequenas oscilações, indicando que o modelo está ajustando os detalhes sazonais;

3. **Platô (épocas 20–50):** as duas perdas permanecem essencialmente constantes, sugerindo que o modelo atingiu sua capacidade máxima de generalização com a arquitetura e os dados disponíveis.

O hiato entre a perda de treino ($\approx 0{,}003$) e a perda de validação ($\approx 0{,}015$) é notável e indicativo de sobreajuste moderado. Com apenas 144 observações e uma rede de 10.000+ parâmetros, algum grau de sobreajuste é esperado e, de facto, inerente ao uso de LSTMs em conjuntos de dados pequenos. A camada de *dropout* atenua esse efeito, mas não o elimina completamente.

6. Resultados

6.1. Previsão na Série Completa

A Figura 6 apresenta as previsões do modelo LSTM ao longo de toda a série, cobrindo os períodos de treino, validação e teste.

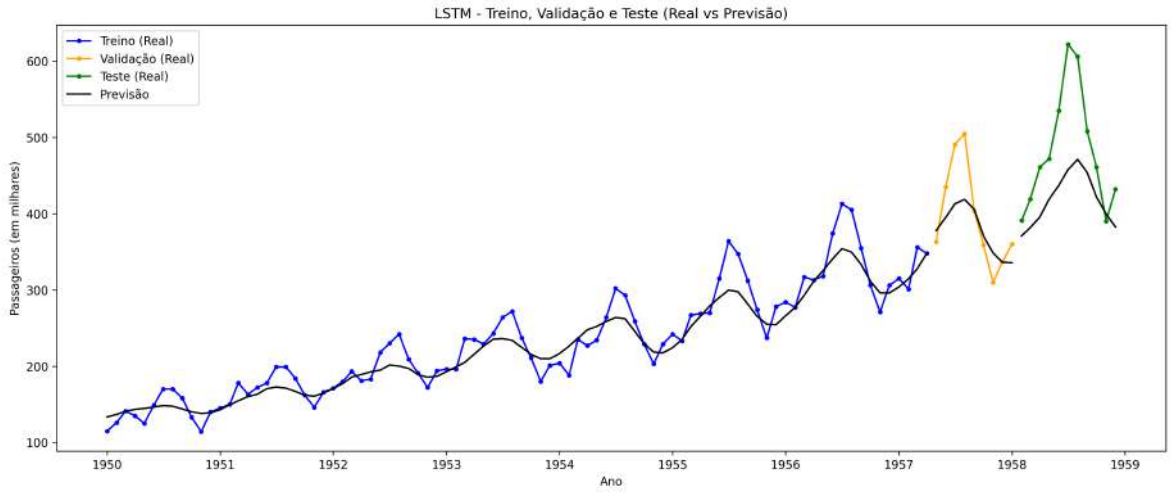


Figure 6. Previsões da LSTM (preto) sobre a série completa, com treino (azul), validação (laranja) e teste (verde). O modelo captura adequadamente a tendência crescente, mas suaviza os picos sazonais, em particular no período de teste.

Observa-se que o modelo aprende bem a tendência de crescimento e consegue reproduzir o padrão sazonal de forma aproximada durante o período de treino e validação. Contudo, no conjunto de teste — os anos de 1959–1960, correspondentes aos valores mais elevados da série — as previsões mostram um comportamento de **suavização excessiva**: a linha preta de previsão é consideravelmente mais suave do que a série real (verde), subestimando os picos de verão e superestimando os vales de inverno. Esse fenômeno é discutido em detalhe na Seção 8.

6.2. Previsão no Conjunto de Teste

A Figura 7 apresenta em detalhe o desempenho do modelo no conjunto de teste, permitindo uma análise mais rigorosa dos erros de previsão.

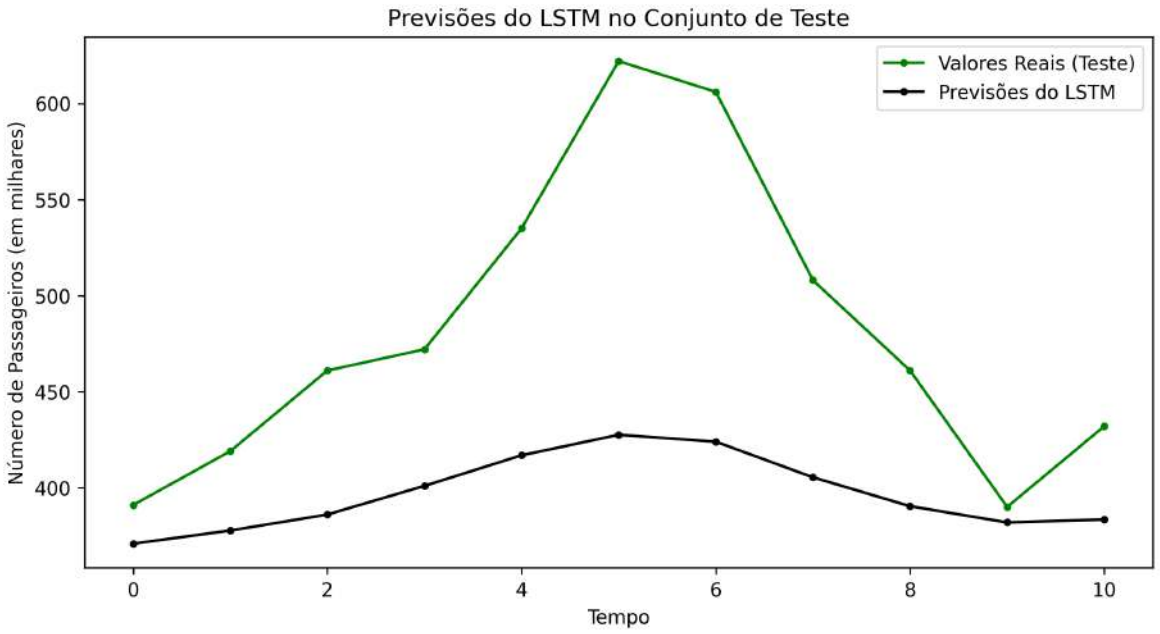


Figure 7. Previsões do LSTM (preto) vs. valores reais (verde) no conjunto de teste. O modelo acompanha a tendência geral crescente, mas atenua significativamente a amplitude das oscilações sazonais, especialmente nos picos de 1960.

A análise do conjunto de teste revela os seguintes padrões:

- O modelo captura a **tendência geral crescente** do período de teste;
- Os **picos sazonais** (pontos de máximo em torno dos passos 5–6) são substancialmente **subestimados**: os valores reais atingem 560–620 mil passageiros, enquanto as previsões ficam em torno de 400.000–450.000;
- Em consequência, a **amplitude sazonal prevista é muito inferior à real**, o que se traduz em erros sistemáticos positivos nos vales e negativos nos picos.

Este padrão de suavização excessiva é uma assinatura típica das redes neurais em regime de dados escassos: o modelo aprende a média ponderada dos padrões vistos no treino, produzindo previsões conservadoras que minimizam o EQM global mas falham em reproduzir as amplitudes extremas.

7. Vantagens do Modelo LSTM

1. **Aprendizado de dependências de longo alcance.** O mecanismo de estado da célula, através das portas de esquecimento seletivo, permite que a LSTM mantenha e acesse informação de centenas de passos anteriores. Isso é uma vantagem fundamental para séries com ciclos de longo período, tendências não

lineares ou mudanças de regime graduais, onde a capacidade de memória limitada das RNNs simples seria insuficiente [6, Capítulo 11.6].

2. **Capacidade de capturar não linearidades.** Um LSTM não impõe nenhuma restrição funcional à relação entre passado e futuro. Ela pode aprender comportamentos assimétricos, interações entre tendência e sazonalidade, e respostas não lineares a choques externos, sem que o analista precise especificá-los a priori [12].

3. **Flexibilidade em entradas multivariadas.** A rede pode receber múltiplas séries como entrada simultaneamente, integrando variáveis exógenas (indicadores económicos, variáveis climatológicas, calendário) de forma natural na camada de entrada, sem necessidade de especificação de funções de transferência [5].

4. **Múltiplas sazonalidades.** Em dados com frequências sub-diárias ou múltiplos padrões cíclicos sobrepostos (e.g., padrão diário, semanal e anual), a LSTM pode aprender todos simultaneamente, desde que o histórico disponível seja suficientemente longo [1].

5. **Robustez a valores em falta e irregularidades.** Ao contrário de alguns métodos clássicos que exigem observações regularmente espaçadas e completas, uma LSTM pode ser adaptada para lidar com séries irregulares e com lacunas, através de arquiteturas especializadas [5].

8. Desvantagens e Limitações

1. **Elevada necessidade de dados.** Uma camada LSTM com 50 unidades e entrada escalar possui mais de 10.000 parâmetros. Estimar com fiabilidade tantos parâmetros requer uma quantidade de dados substancialmente maior do que a disponível no conjunto Airline (144 observações). Isso explica diretamente o sobreajuste observado na Figura 5 e a suavização excessiva das previsões nas Figuras 6 e 7 [8].

2. **Sobreajuste e suavização das amplitudes.** Como se observa claramente nos resultados, o modelo tende a subestimar picos e sobrestimar vales. Isso decorre do facto de que minimizar o EQM global numa série pequena favorece previsões conservadoras próximas da média, em detrimento da fidelidade às flutuações extremas. Regularização adicional (L2, *dropout* variacional) pode mitigar, mas não eliminar, este problema [8].

3. **Seleção de hiperparâmetros sem critério formal.** A escolha do número de camadas, unidades ocultas, comprimento da janela L, taxa de *dropout*, tamanho

do lote e taxa de aprendizado não dispõe de critérios análogos ao AIC/BIC ou às análises ACF/PACF dos modelos clássicos. A seleção é feita por busca em grade ou aleatória, o que é computacionalmente intensivo e sem garantias de otimalidade [8].

4. **Não identificabilidade e irreprodutibilidade.** Diferentes inicializações aleatórias podem produzir modelos distintos, o que pode comprometer a reprodutibilidade dos resultados — uma limitação séria em contextos científicos onde a transparência e a replicabilidade são exigidas [8].

5. **Opacidade do modelo (*black box*).** Os pesos aprendidos não têm interpretação estatística direta. Não é possível extrair, por exemplo, a ordem de integração da série, a estrutura de autocorrelação ou os coeficientes de uma equação de previsão interpretável. Isso dificulta a comunicação dos resultados a utilizadores não técnicos e a validação por especialistas de domínio.

6. **Custo computacional.** O treinamento de redes LSTM, mesmo de porte moderado, é substancialmente mais lento do que a estimação de modelos clássicos. Para conjuntos de dados grandes ou arquiteturas profundas, a utilização de GPU torna-se necessária [8].

9. Como Melhorar o Modelo

9.1. Janela Deslizante Maior

Com $L = 12$, o modelo recebe exatamente um ciclo sazonal. Aumentar a janela para $L = 24$ ou $L = 36$ fornece ao modelo mais contexto histórico, permitindo-lhe identificar melhor as flutuações sazonais e a aceleração da tendência. No entanto, essa escolha implica um aumento do número de parâmetros efetivos a serem estimados e, consequentemente, requer uma quantidade maior de dados para evitar sobreajuste e garantir uma estimação robusta do modelo.

9.2. Arquiteturas Mais Profundas e Regularização

Empilhar duas camadas LSTM (LSTM empilhada, *Stacked LSTM*) pode aumentar a capacidade do modelo para séries com padrões hierárquicos. Combinar isso com *dropout* permite mitigar o problema de **sobreajuste** (*overfitting*), que ocorre quando o modelo se ajusta excessivamente aos dados de treino, capturando não apenas o padrão subjacente, mas também o ruído específico da amostra [8].

Nessas circunstâncias, o modelo apresenta erro muito baixo no conjunto de treino, mas desempenho inferior em dados não observados, indicando baixa capacidade de

generalização. O *dropout* atua como um mecanismo de regularização, desativando aleatoriamente uma fração das unidades durante o treinamento, o que reduz a dependência entre neurônios e força o modelo a aprender representações mais robustas.

9.3. LSTM Bidirecional

As *Bidirectional LSTMs* (BiLSTMs) constituem uma extensão da arquitetura LSTM padrão, na qual a sequência de entrada é processada em duas direções: para a frente (passado → futuro) e para trás (futuro → passado). Em vez de utilizar apenas informação passada, o modelo incorpora contexto de ambos os lados de cada instante temporal.

Uma BiLSTM é composta por duas camadas LSTM paralelas. Uma camada processa a sequência na ordem temporal natural, enquanto a outra a processa em ordem inversa. Os estados ocultos resultantes são então combinados (tipicamente por concatenação), produzindo uma representação mais rica da sequência em cada passo de tempo.

Este processamento bidirecional é particularmente útil em tarefas nas quais a interpretação de uma observação depende tanto do contexto anterior quanto do posterior. Ao explorar informação em ambas as direções, as BiLSTMs conseguem capturar dependências temporais de forma mais completa do que as LSTMs unidirecionais.

No entanto, em previsão de séries temporais clássica, a sua utilização é limitada. Como as observações futuras não estão disponíveis no momento da previsão, a incorporação de informação ”para trás” pode introduzir *data leakage* se não for tratada cuidadosamente. Por essa razão, as BiLSTMs são mais frequentemente utilizadas em tarefas de modelação de sequências, como processamento de linguagem natural, do que em cenários estritamente causais de previsão [8].

9.4. Mecanismos de Atenção e Transformers

O mecanismo de atenção (*attention*) permite que o modelo atribua pesos diferentes às observações passadas, em vez de tratar todo o histórico de forma uniforme. Em cada passo de previsão, o modelo pode concentrar-se seletivamente nas partes mais relevantes da sequência de entrada, capturando melhor o contexto necessário para gerar a saída.

Essa capacidade é particularmente útil em sequências longas, nas quais nem todas as observações passadas são igualmente informativas para a previsão atual. Em modelos recorrentes clássicos, como as LSTMs, toda a informação histórica é comprimida no estado oculto, o que pode levar à perda de detalhes importantes. O mecanismo de

atenção contorna essa limitação ao permitir acesso direto a diferentes pontos do histórico.

Em aplicações de séries temporais, isso significa que o modelo pode, por exemplo, atribuir maior importância a observações associadas a padrões sazonais ou a eventos específicos relevantes para a previsão corrente. Como consequência, obtêm-se previsões mais precisas e consistentes, especialmente em contextos com dependências de longo alcance [8].

10. Adequação de Modelos: LSTM vs. Métodos Clássicos

Os resultados obtidos com o conjunto Airline ilustram de forma concreta uma tensão fundamental na análise de séries temporais modernas: a flexibilidade de um modelo não paramétrico é uma vantagem quando há dados suficientes e padrões complexos, mas torna-se uma desvantagem quando os dados são escassos e o padrão dominante é simples e bem estruturado.

O conjunto Airline tem apenas 144 observações, tendência monotónica e uma única sazonalidade anual regular. Nesse contexto, a LSTM precisa de muitos parâmetros para aprender o que um modelo mais parcimonioso pode capturar com poucos. O sobreajuste e a suavização dos picos sazonais são consequências diretas dessa desproporção entre a complexidade do modelo e a quantidade de dados disponíveis.

Quando as LSTMs são mais apropriadas?

- Padrões complexos ou não lineares que modelos paramétricos não conseguem capturar;
- Conjuntos com múltiplas séries relacionadas, onde o modelo pode aprender representações partilhadas (*multi-task learning*);
- Dados multivariados com interações complexas entre variáveis;
- Contextos onde a acurácia preditiva é prioritária e a interpretabilidade é secundária.

Em contrapartida, para séries regulares com poucos dadoes e com estrutura bem compreendida, os métodos estatísticos clássicos continuam a ser particularmente eficazes. A sua natureza parcimoniosa permite capturar padrões dominantes com poucos parâmetros, ao mesmo tempo que oferecem interpretabilidade, intervalos de previsão bem fundamentados e critérios formais de seleção de modelos.

Ainda assim, a literatura recente tem demonstrado que, em determinados contex-

tos, a combinação de abordagens pode ser vantajosa. Em particular, modelos híbridos que integram estruturas clássicas com redes neuronais têm apresentado desempenho superior em tarefas de previsão, ao explorar simultaneamente a eficiência estatística dos modelos paramétricos e a flexibilidade dos métodos de aprendizado profundo [11].

Deste modo, mais do que uma substituição de paradigmas, o estado da arte aponta para uma integração entre abordagens. Compreender as vantagens, limitações e condições de aplicabilidade de cada uma constitui, portanto, uma competência essencial para o analista moderno.

11. Conclusão

Este relatório apresentou uma discussão detalhada e crítica das redes LSTM aplicadas à previsão de séries temporais, ilustrada com resultados empíricos próprios sobre o conjunto *International Airline Passengers*. As principais conclusões são:

- O mecanismo de portas da LSTM resolve o problema do gradiente evanescente das RNNs clássicas, permitindo o aprendizado de dependências temporais de longo alcance;
- No conjunto Airline, o modelo aprende corretamente a tendência crescente, mas suaviza excessivamente as amplitudes sazonais — uma limitação diretamente associada ao regime de dados escassos (144 observações, mais de 10.000 parâmetros);
- O sobreajuste moderado, evidenciado pelo hiato entre perdas de treino e validação, é um resultado esperado e documentado na literatura para LSTMs aplicadas a séries temporais curtas;
- As principais limitações do modelo — opacidade, ausência de incerteza formal, custo computacional e necessidade de grandes volumes de dados — são contrabalançadas pelas suas vantagens em contextos de maior complexidade e escala;
- Extensões como mecanismos de atenção ou modelos híbridos representam direções promissoras para superar as limitações identificadas.

Do ponto de vista desta disciplina, as redes LSTM constituem um complemento valioso ao arcabouço clássico de Morettin and Toloi [10]: oferecem uma perspectiva radicalmente diferente sobre o problema de previsão, com pontos fortes distintos e limitações complementares. A capacidade de transitar entre os dois paradigmas — sabendo quando cada um é mais adequado — é a marca de uma prática rigorosa e informada em análise de séries temporais.

Referências

[1] Kasun Bandara, Christoph Bergmeir, and Hansika Hewamalage. "LSTM-MSNet: Leveraging forecasts on sets of related time series with multiple seasonal patterns". en. In: *IEEE Trans. Neural Netw. Learn. Syst.* 32.4 (Apr. 2021), pp. 1586–1599.

[2] George E. P. Box, Gwilym M. Jenkins, and Gregory C. Reinsel. *Time series analysis*. Fourth. Wiley Series in Probability and Statistics. Forecasting and control. John Wiley & Sons, Inc., Hoboken, NJ, 2008, pp. xxiv+746. isbn: 978-0-470-27284-8. doi: 10.1002/9781118619193. url: https://doi-org.tum-eaccess.de/10.1002/9781118619193.

[3] Ovidiu Calin. *Deep learning architectures—a mathematical approach*. Springer Series in the Data Sciences. Springer, Cham, [2020] ©2020, pp. xxx+760. isbn: 978-3-030-36720-6; 978-3-030-36721-3. doi: 10.1007/978-3-030-36721-3. url: https://doi-org.tum-eaccess.de/10.1007/978-3-030-36721-3.

[4] François Chollet et al. *Keras: LSTM layer*. 2015. url: https://keras.io/api/layers/recurrent_layers/lstm/ (visited on 04/15/2026).

[5] Nazanin Fouladgar and Kary Främling. "A novel LSTM for multivariate time series with massive missingness". en. In: *Sensors (Basel)* 20.10 (May 2020), p. 2832.

[6] Arash Gharehbaghi. *Deep Learning in Time Series Analysis*. May 2023. isbn: 9780429321252. doi: 10.1201/9780429321252.

[7] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9 (Nov. 1997), pp. 1735–1780. doi: 10.1162/neco.1997.9.8.1735.

[8] Moez Krichen and Alaeddine Mihoub. "Long Short-Term Memory Networks: A Comprehensive Survey". In: *AI* 6.9 (2025). issn: 2673-2688. doi: 10.3390/ai6090215. url: https://www.mdpi.com/2673-2688/6/9/215.

[9] Xiaolei Liu, Zi Lin, and Ziming Feng. "Short-term offshore wind speed forecast by seasonal ARIMA - A comparison against GRU and LSTM". In: *Energy* 227 (2021), p. 120492. issn: 0360-5442. doi: https://doi.org/10.1016/j.energy.2021.120492. url: https://www.sciencedirect.com/science/article/pii/S0360544221007416.

[10] Pedro Alberto Morettin and Clélia Maria de Castro Toloi. *Análise de séries temporais*. Edgard Blucher, 2006.

[11] Agbessi Akuété Pierre et al. "Peak Electrical Energy Consumption Prediction by ARIMA, LSTM, GRU, ARIMA-LSTM and ARIMA-GRU Approaches". In: *Energies* 16.12 (2023). issn: 1996-1073. doi: 10.3390/en16124739. url: https://www.mdpi.com/1996-1073/16/12/4739.

[12] Sandeep Yadav. "A Comparative Study of ARIMA, Prophet and LSTM for Time Series Prediction". In: *Journal of Artificial Intelligence, Machine Learning and Data Science* 1.1 (2022), pp. 1813–1816. doi: 10.51219/JAIMLD/sandeep-yadav/402.

Para auxiliar na tradução e correção gramatical do texto, recorreu-se às ferramentas DeepL e Grammarly.

Arquivos Supplementares

Fluxograma do método do Análise

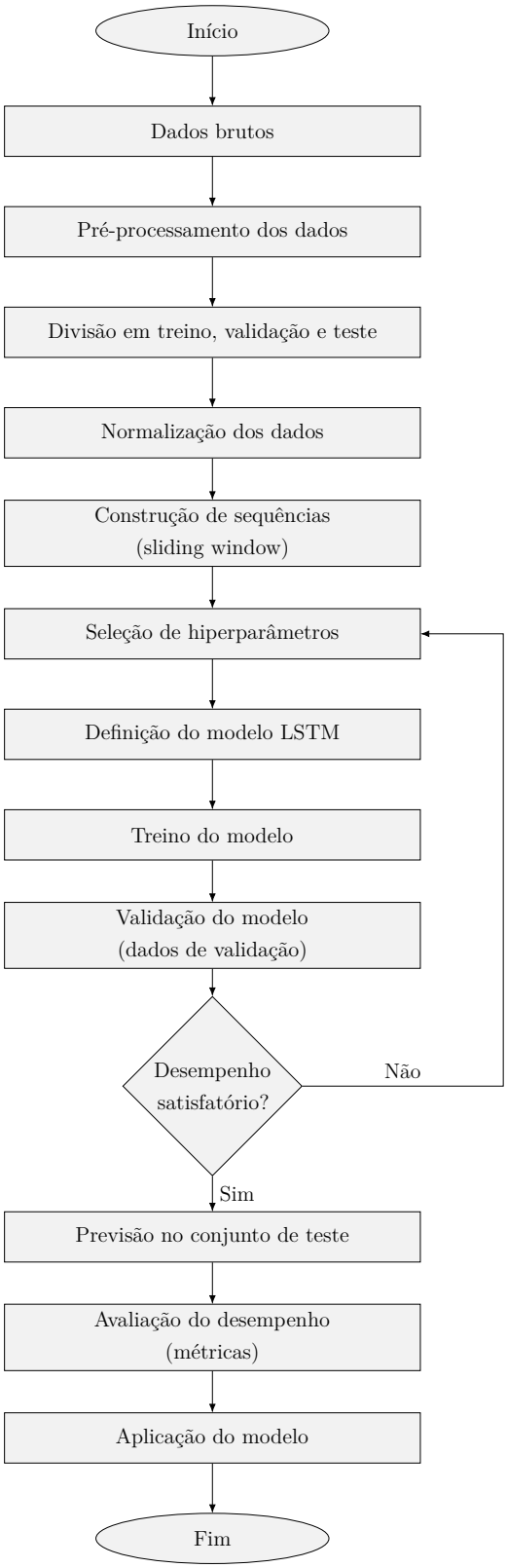


Figure 8. Fluxograma do método de análise

Fluxograma do algoritmo implementado no programa

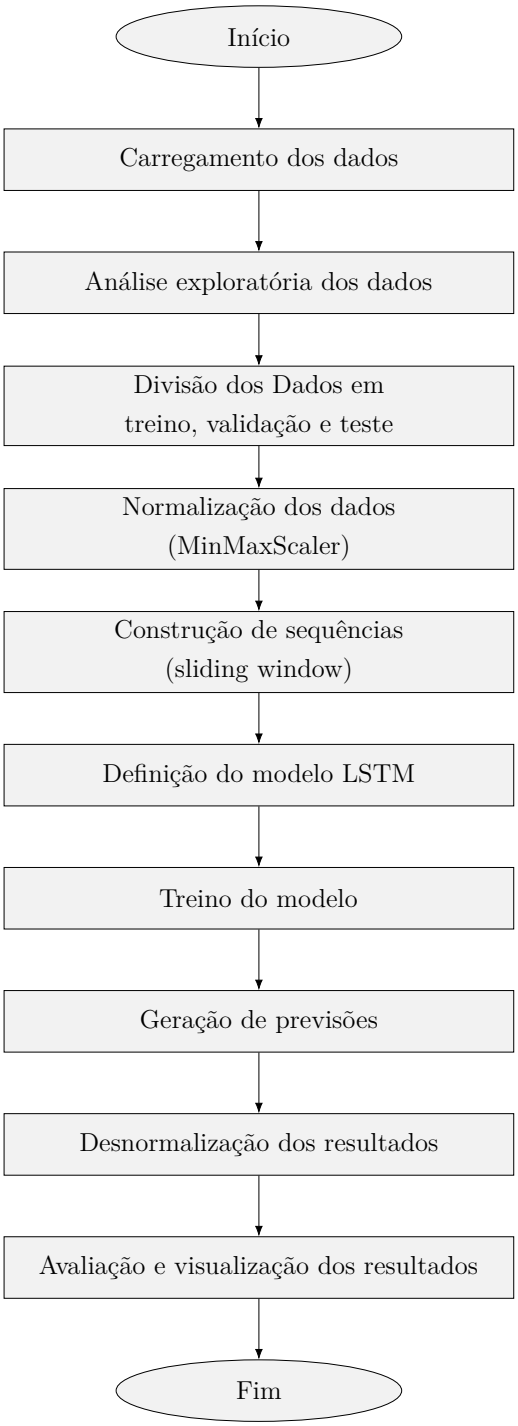


Figure 9. Fluxograma do algoritmo implementado